



WATCHDOG

Intelligent Observability & Event Watchdog

Multi-tenant · API-first · Production-grade · AI-powered Anomaly Detection

200+ Tests

89%+ Coverage

6 Anomaly Types

Precision: 1.000

Built with Claude Code (Sonnet 4.6) · Architect-directed Vibe Coding Methodology



WATCHDOG

Why Watchdog Exists

The Problem



Silent Failures

Services crash silently. Error rates spike. Latency degrades. No one notices until customers complain.



Alert Fatigue

Static thresholds generate noise. Engineers ignore alerts. The one real incident gets buried in false positives.



Integration Complexity

Every team has different log formats, databases, deployment patterns. No single tool connects all of them.

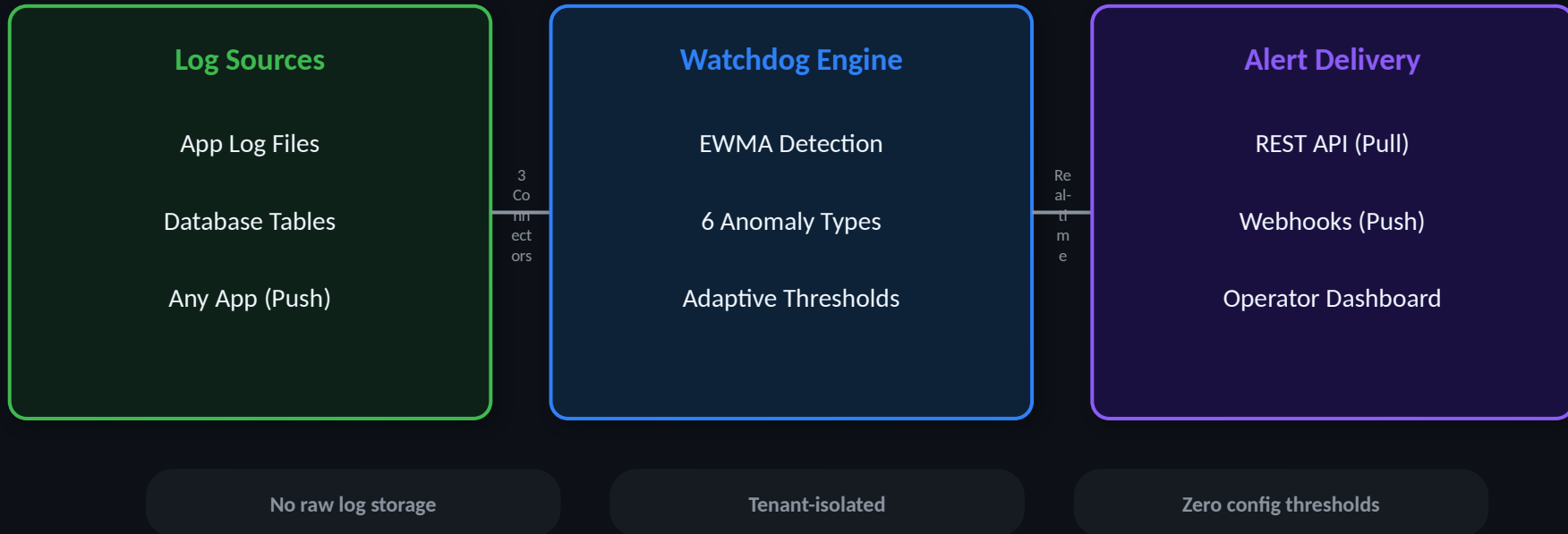
Watchdog solves all three — adaptive AI detection, intelligent thresholds, plugin connector architecture



WATCHDOG

End-to-end data flow

How Watchdog Works

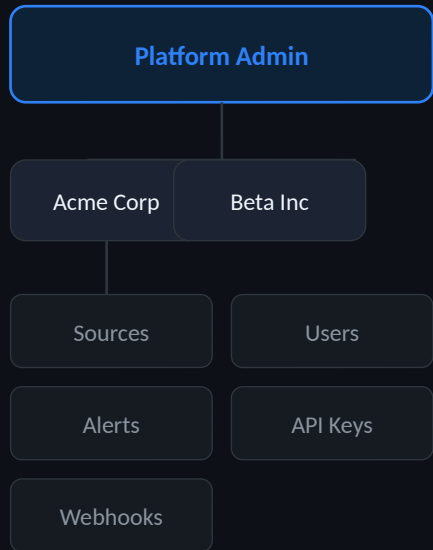




WATCHDOG

Complete data isolation by design

Multi-Tenant SaaS Architecture



Complete Data Isolation

- ✓ `tenant_id` on every database table — no exceptions
- ✓ TenantContext injected into every API call (verified from JWT or API key, never raw input)
- ✓ Cache keys namespaced: `ewma:{tenant_id}:{source_id}`
- ✓ No cross-tenant data leakage possible at application layer
- ✓ PostgreSQL Row Level Security — documented upgrade path for DB-layer defense-in-depth

Tenant A can never see Tenant B's data — by design, not by convention



AI Detection: EWMA Algorithm

Algorithm

$$\text{EWMA}_t = \alpha \times x_t + (1 - \alpha) \times \text{EWMA}_{\{t-1\}}$$

$$\text{Var}_t = \alpha \times (x_t - \text{EWMA}_{\{t-1\}})^2$$

$$+ (1 - \alpha) \times \text{Var}_{\{t-1\}}$$

$$\text{Upper} = \text{EWMA}_t + \text{sens} \times \sqrt{\text{Var}_t}$$

$$\alpha = 0.3 \cdot \text{WARNING} = 2.5\times \cdot \text{CRITICAL} = 5.0\times$$

Used by: [AWS CloudWatch](#) · [Netflix Atlas](#) · [Google SRE](#)

Why EWMA over alternatives?

✗	Static threshold	Manual tuning, misses relative spikes
✗	Rolling average	Lags badly on real spikes
✗	Z-score	Assumes stationary distribution
✗	LSTM / Neural Net	Needs training data, not explainable
✗	Prophet	External dependency, slow retraining
✓	EWMA	Adapts, O(1), explainable, proven

Precision: 1.000

Recall: 1.000

FPR: 0.000



WATCHDOG

Covering the full spectrum of production failure modes

Six Anomaly Types Detected



ERROR_RATE_SPIKE

Sudden error burst above EWMA upper bound

Severity: WARNING / CRITICAL

Auto-resolves after 2 clean windows



SUSTAINED_ELEVATION

Error rate above baseline for >10 minutes

Severity: WARNING / CRITICAL

Systemic vs transient distinction



SERVICE_SILENCE

Active service stops emitting logs entirely

Severity: CRITICAL always

Catches crashes invisible to error-rate



LATENCY_SPIKE

Response time EWMA upper bound breach

Severity: WARNING / CRITICAL

Requires latency_field in source config



NOVEL_ERROR

Error pattern unseen in last 24 hours

Severity: WARNING

Bloom filter + 24h TTL fingerprinting



CASCADE

3+ services spiking within 5-minute window

Severity: CRITICAL always

Infrastructure-level failure signal



Plugin Connector Architecture

File Connector

For apps writing to log files

- Byte-level cursor: inode + byte offset
- Rotation: file shrink (primary) + inode (secondary)
- Formats: JSON · logfmt · plaintext
- watchdog lib + polling fallback (NFS-safe)
- Binary mode reads (Windows-compatible)

DB Connector

For apps logging to databases

- `SELECT WHERE id > last_seen_id LIMIT 500`
- $O(\log n)$ with B-tree index — no full table scan
- Supports: PostgreSQL · MySQL · SQLite
- Validates index exists on source setup
- `asyncio.to_thread()` for sync SQLAlchemy calls

Push Connector

For greenfield applications

- `POST /api/v1/ingest` (single entry)
- `POST /api/v1/ingest/batch` (up to 500)
- X-API-Key authentication (scope: ingest)
- Pydantic validation on every field
- Rate limited: 100 req/min per key

Adaptive: ACTIVE (5s) → IDLE (30s) → BACKOFF (60s) · New connector = one new file, zero changes to existing code



Security Architecture

1

TIER 1 — NEVER STORED

API key plaintext

Generated · Returned once · Forgotten

2

TIER 2 — HASHED (ONE-WAY)

User passwords → bcrypt cost 12

API key lookup → SHA-256

Refresh tokens → SHA-256

3

TIER 3 — FERNET ENCRYPTED (TWO-WAY)

Webhook secrets (retrieve to sign HMAC)

DB connection strings (retrieve to connect)

Encryption key in .env — never in DB

Key Format

`wdog_live_XXXXXXXXXXXXXXXXXX`

`wdog_test_XXXXXXXXXXXXXXXXXX`

Prefix enables automated secret scanning in git history & CI/CD

Additional Controls

- ✓ JWT: RS256 asymmetric · 15min access token
- ✓ Refresh: httpOnly · Secure · SameSite=Strict
- ✓ Rate limiting: slowapi on auth + ingest endpoints
- ✓ No stack traces in API error responses
- ✓ Sensitive headers redacted from all log output
- ✓ Zero-downtime key rotation (24h grace period)



API-First Design

GROUP A — Ingest

API Key: ingest

POST /api/v1/ingest

POST /api/v1/ingest/batch

GROUP B — Alerts

API Key: alerts:read

GET /api/v1/alerts

GET /api/v1/alerts/{id}

POST /{id}/acknowledge

GROUP C — Webhooks

API Key: webhooks:manage

POST /api/v1/admin/webhooks

GET /api/v1/admin/webhooks

DELETE /admin/webhooks/{id}

GROUP D — Sources

JWT: Tenant Admin

POST /api/v1/admin/sources

GET /api/v1/admin/sources

PATCH/DELETE /sources/{id}

GROUP E — Keys

JWT: Tenant Admin

POST /api/v1/admin/keys

POST /keys/{id}/rotate

DELETE /keys/{id}

GROUP F — Platform

JWT: Platform Admin

POST /api/v1/platform/tenants

GET /api/v1/platform/tenants

GROUP G — Health

None required



WATCHDOG

HMAC-signed · 3× retry · auto-disable after 10 failures

Webhook Push Delivery

1

Anomaly Detected

2

Build v1.0 JSON Payload

3

Compute HMAC-SHA256 Signature

4

POST to consumer webhook_url

5

Log to webhook_events table

On failure: retry 3× · 2s / 4s / 8s backoff · Auto-disable after 10 failures

Delivery Headers

X-Watchdog-Signature: sha256=<hex>

X-Watchdog-Delivery-ID: <uuid>

X-Watchdog-Event: anomaly.detected

Consumer Verification (Python)

```
import hmac, hashlib
```

```
def verify(payload, secret, sig):
```

```
    exp = hmac.new(
        secret.encode(), payload,
        hashlib.sha256).hexdigest()
```

```
    return hmac.compare_digest(
        f'sha256={exp}', sig)
```



Data Architecture Decisions

NOT STORED

Raw log entries from external sources

1000 logs/min × 10 services = 14.4M rows/day
We are a detection service, not a log store

STORED — Anomalies only

- Current value vs EWMA baseline
- Analysis window timestamps
- Top 3 representative error messages
- Full EWMA params at detection time
- CASCADE: all contributing service IDs

SQLite default · PostgreSQL upgrade = <code>openalerts</code> never auto-deleted

Database — 11 Tables

tenants

users

api_keys

refresh_tokens

log_sources

source_state

ewma_state

anomaly_alerts

webhook_events

request_log

system_config

- ✓ UUIDs as PKs (prevents enumeration attacks)
- ✓ tenant_id FK on every user-facing table
- ✓ Indexes on all WHERE/ORDER BY columns
- ✓ Soft deletes (deleted_at) — never hard delete
- ✓ Alembic migrations — no DDL in app code
- ✓ Configurable retention (default 30 days)
- ✓ Open alerts never auto-deleted



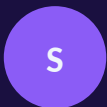
Four Roles • Four Views



PLATFORM ADMIN

/platform-admin

- Creates tenant accounts
- Platform health overview
- Runtime log level control
- Suspend/reactivate tenants



TENANT ADMIN

/admin

- Configure log sources
- Manage users & retention
- Generate API keys
- Tune EWMA per source



TENANT OPERATOR

/dashboard

- View alert feed & trends
- Acknowledge alerts
- Filter by severity/service
- Cannot access admin config



API CONSUMER

/consumer

- Generate/rotate/revoke keys
- Register webhook subscriptions
- View delivery history
- Cannot see other users' data



Architect-Directed Vibe Coding

1

Architecture Phase

5 docs before any code
CLAUDE.md · FEATURES.md · ARCHITECTURE.md
DECISIONS.md · tasks/todo.md

2

ARCHITECT PAUSE Pattern

Prompt 1: rules only, stops Claude
Prompt 2: architecture + comprehension
check required before GO

3

Module-by-Module

15 modules · Complete → Test → Verify
Explicit approval before each next step

4

Self-Improvement Loop

tasks/lessons.md: mistake · root cause
· prevention rule — 8 lessons captured

15

Modules built

200+

Tests passing

89%+

Code coverage

8

Bugs caught & fixed

18

Architecture decisions

~8 hrs

Total build time

"The quality of AI output is determined by the quality of the specification you give it."



18 Architectural Decisions

DATA ARCHITECTURE

Decision	Choice	Tradeoff
Log storage	Anomalies only	No raw log search
Database	SQLite + PG path	Write concurrency limits
Caching	In-process + Redis abstraction	Cache lost on restart
Retention	30-day configurable	Open alerts exempt

SECURITY

Decision	Choice	Tradeoff
API key storage	SHA-256 hash	One-way, unrecordable
Webhook secrets	Fernet encrypted	Must retrieve to sign
Key format	wdog_live_prefix	Automated scanning
Auth	JWT + API keys	Two auth systems

DETECTION

Decision	Choice	Tradeoff
Algorithm	EWMA + adaptive variance	No seasonality
Anomaly types	6 distinct types	6 algorithms to test
Connectors	Plugin ABC, 3 types	3x test surface
Polling	Adaptive ACTIVE/IDLE/BACKOFF	State machine complexity

INFRASTRUCTURE

Decision	Choice	Tradeoff
Product model	Multi-tenant SaaS	Higher initial complexity
Data isolation	App layer + RLS path	DB-layer safety
API versioning	URL path /api/v1/	Version baked in
Frontend	Jinja2 + Alpine.js	JS not Python on App-layer only
Tenant isolation	TenantContext DI	
DB polling	High-water mark	Requires integer PK



Honest Limitations & Phase 2 Roadmap

Known Limitations

SQLite write concurrency

Single-writer limit under sustained high load → Change connection string → PostgreSQL

No seasonality modeling

EWMA adapts but misses weekly patterns → Prophet integration for seasonal decomp

No RLS in SQLite

Application-layer isolation only → PostgreSQL RLS for DB-layer defense-in-depth

In-process cache

Lost on restart, single-instance only → `CACHE_BACKEND=redis` in `.env` (abstraction ready)

No email verification

Instant account creation → SendGrid integration, verify-then-activate

Phase 2 Roadmap

» LLM Enrichment (Claude claude-sonnet-4-6)

ai_enrichment field in output contract (currently null)
Plain English explanation + likely root causes
EWMA detection never depends on LLM availability

» Next.js Frontend

FastAPI API is fully decoupled
Next.js consumes `/api/v1/` with zero backend changes
Jinja2 templates retired incrementally

» Kafka Connector

New file implementing LogSourceConnector ABC
Zero changes to existing connector code

» Multi-region Deployment

PostgreSQL + Redis + load balancer
Application code unchanged

WATCHDOG

Production-ready from day one

DETECT

- 6 anomaly types
- EWMA AI algorithm
- Precision: 1.000
- Recall: 1.000

ISOLATE

- Multi-tenant SaaS
- TenantContext everywhere
- 200+ tests passing
- Cross-tenant isolation verified

DELIVER

- REST API (pull)
- Webhooks (push)
- HMAC-signed payloads
- 3× retry with backoff

OPERATE

- 4 role-based UI views
- Runtime log level control
- Configurable retention
- Self-service API keys

Explore the architecture: [README.md](#) · [ARCHITECTURE.md](#) · [DECISIONS.md](#) · [FEATURES.md](#) · [ARCHITECT_PROMPTS.md](#)

Built with deliberate architectural thinking. Every decision documented. Every tradeoff considered.